ChatGPT

Phase 1 Research Plan – Gig-Worker Income Protection

We focus on **food-delivery and quick-commerce riders** (e.g. Zomato/Swiggy/Blinkit). These gig workers typically log very long hours for modest pay: surveys show full-time riders (~8–10 hr/day, 26 days/month) gross about ₹26,000–30,000 per month (₹21k–25k net after fuel/maintenance) [1] [2]. In practice, many delivery workers report earnings only ~₹800–1,200 per day after 12–16 hours of work [3] [4]. Pay is a mix of **base fees** (₹20–50 per order plus distance surcharges) and **incentives** for meeting daily order targets [5]. Peak meal hours (7–10 pm) yield the highest bonuses [6].

- **Earnings pattern:** Typical gig riders earn several hundred rupees per trip (often ₹20–50 base + bonuses) but rely on completing many orders. A Zomato analysis found one partner made ~₹65/hour (logged) and ₹38 per order on New Year's Eve [7]. Weekly and daily income can swing widely: one rider's weekly gross ranged from ₹300/day in slow weeks to ~₹1,000/day in busy weeks [4].

- **Expenses:** Riders spend ~20% of gross on fuel, phone data, and bike upkeep [1] [8]. For example, one rider said he earned ₹1,000–1,200/day but spent ₹300–400 on petrol [3]. These slim margins mean any day off work can be a serious hardship.

- **Persona:** We will target **Food Delivery riders** (e.g. Swiggy/Zomato) in urban India. They are mostly male, often migrant workers, and lack formal benefits. They depend entirely on daily wages and have no paid leave or downtime buffer [9] [3].

Disruption Events & Income Loss

Gig incomes are cut off by external events that halt work. Key **environmental** triggers include:

- **Heavy Rain / Floods:** Intense rainwater floods streets and slows or stops delivery. Riders report that "waterlogging… causes accidents; either the bike gets damaged or I get wounded, and I lose the day's pay" [3]. Indeed, during extreme downpours (e.g. NCR floods), traffic jams can turn 30 min trips into 6-hour ordeals [10]. In such cases, orders vanish. We will define parametric triggers like *"daily rainfall > X mm"* or flood warnings. For example, a threshold of ~40–50 mm in 24h could trigger coverage.

- **Extreme Heat / Heatwaves:** Very high temperatures (e.g. ≥45 °C) make outdoor work unsafe. A World Economic Forum report notes that parametric insurance has been used in India to cover outdoor workers' income losses during heat waves [11]. We will consider triggers like *"daily max temp > 45°C"*.

- **Air Pollution:** Severe smog/AQI spikes also disrupt work. High pollution forces advisories to stay indoors. Research shows air pollution is effectively "a regressive tax" on poor, outdoor workers,

reducing their productivity and income [12]. We can set triggers such as *"AQI > 300"* or government "Severe Pollution" alerts to payout losses during those days.

- **Social/Regulatory Disruptions:** Government actions (curfews, lockdowns, election-related traffic bans) can instantly shut down city movements. For example, nighttime curfews or strike calls cancel orders city-wide. We plan to include triggers like official curfew announcements or public event declarations. These may be detected via news feeds or city API alerts.

- **Platform Outages:** System failures (app/server downtime, payment gateway issues) also cause losses. For instance, if a delivery app is down for >30 min in a zone, we can auto-trigger compensation. While real outage data may require collaboration with platforms, we'll simulate it.

"**Income loss only**" is covered. We **exclude** vehicle, health or accident coverage. If a rider's bike breaks or they get injured, our policy does *not* pay – only lost wages due to uncontrollable external events trigger a payout [13] [14].

Figure: A delivery rider braves heavy rain (Unsplash/Kenny) – parametric insurance could automate pay-outs when rain exceeds safety thresholds [14] [3].

Parametric Insurance Model

We will build an **event-driven insurance** model: payouts occur automatically when predefined conditions ("parametric triggers") are met. This is exactly suited to gig income loss: instead of claims investigation, the system monitors data feeds and *knows* when an event occurs.

- **Triggers & Payouts:** Examples of parametric triggers:
- Rainfall: e.g. >40 mm in last 24h → ₹X payout.
- Temperature: e.g. max >45°C → ₹Y.

- AQI: e.g. AQI >400 (Severe) → ₹X.
- Curfew announcement in city Z → ₹Y.
- App outage >30min in area A → ₹X.

If any condition is met while a worker is scheduled, we credit a fixed amount (based on plan) to the worker's account immediately. This aligns with industry practice: parametric payouts "are triggered when specific conditions are met, such as certain levels of rainfall or wind speed" [14], ensuring rapid, transparent relief.

- **Weekly Pricing:** Premiums are charged weekly (not monthly/annual) to match gig incomes. We'll design tiered plans (e.g. Basic/Standard/Premium) with weekly fees in the range of a few tens of rupees. For instance, if average rider earns ~₹5,000–8,000/week, we might price Basic ~₹30/week (max payout ₹500), Standard ~₹60/week (max ₹1200), and Premium ~₹100/week (max ₹2500). These numbers will be calibrated to approximate risk (based on how often triggers occur). Studies show gig riders net around ₹500–1000/day [3] [4], so a few hundred-rupee payout can meaningfully cover a lost day.

- **AI Risk Assessment:** We'll use ML to make premiums dynamic and fair. Inputs like city, historical weather patterns, and worker's operating hours can feed a risk model that sets individualized premiums each week. For example, riders in highly flood-prone or pollution-heavy zones may pay slightly more. We'll train models on historical weather vs. outage data to predict disruption probability per region. This leverages AI to balance the pool's risk.

- **Transparency & Speed:** Because the rules are defined upfront (e.g. "rain >40 mm"), payouts are fast and objective. This addresses traditional insurance gaps: only ~57% of sharing-economy participants even have insurance [15], and parametric solutions specifically aim to fill this "coverage gap" for income loss [13] [14].

Figure: Flooded streets stop deliveries. Studies highlight how outdoor workers lose pay in such conditions [3] [12]. Our system would use weather triggers (rain, AQI, etc.) to automate pay-outs when these events occur.

Data Integration & APIs

To detect disruptions in real time, we'll integrate external data feeds:

- **Weather & Pollution APIs:** We'll use services like **OpenWeatherMap**. OpenWeather's APIs (free tier available) provide current/forecast data on temperature, rainfall, and air quality index [16]. For example, its **Air Pollution API** returns real-time AQI and pollutant levels [16]. We can query rainfall or temperature thresholds per city.

- **Traffic & Location APIs:** To catch floods or road blockages, we can query traffic data. For India, providers like **MapmyIndia** offer real-time traffic/flow data via APIs [17], covering congestion and estimated travel times. These feeds can signal city-wide gridlock (e.g. during heavy rain or curfew), supplementing the weather trigger.

- **Platform Activity (Mock):** Ideally, we would access the delivery platform's backend data (order counts, cancellations). For a hackathon, we'll simulate this. For example, if no new orders are dispatched for 30+ minutes in an area, we treat it as a disruption. In future, an actual partnership with a platform could use their internal APIs or webhooks.

- **Payment Gateway:** For automated payouts, we'll integrate a payment API. Services like **Razorpay** or **Stripe** have sandbox modes for testing [18]. Using Razorpay's test credentials, we can programmatically push payouts (via UPI or wallet) in the demo. When a trigger fires, our backend calls the payment API to release funds to the worker's account.

Fraud Detection

To prevent abuse, we'll build an **AI-based fraud monitoring** layer. Possible scams include fake location claims or duplicate claims by colluding workers. We'll use:

- **Location Validation:** Verify the worker's GPS history. For instance, if a payout is due for a rain-trigger in Delhi but the rider's phone GPS shows them offline or in another city, flag the claim. Location/time logs (e.g. from the delivery app) can be cross-checked. Geofencing tools (like Radar) can detect "spoofed locations" and inconsistent claim patterns [19].

- **Anomaly Detection:** Using machine learning, we can model normal patterns of work and claims. For example, if a rider never claimed a payout before but suddenly files multiple claims in one week, or if many riders in the same area claim exactly the same payout amounts repeatedly, those patterns can be detected as anomalies. Advanced ML models (e.g. Autoencoders or tree ensembles) are effective at spotting such irregularities in insurance claims data [20] [21].

- **Duplicate Prevention:** The system will allow only one payout per event per worker (and track events globally). If the same trigger (e.g. "Delhi rain event of July 3") already paid one week's worth, duplicate submissions will be ignored automatically.

In essence, by correlating **geo-data, time stamps, and claim histories**, we can filter out fraud. As one source notes, location and time logs are powerful fraud indicators: "Identify spoofed locations, duplicate accounts, or inconsistent claims using location and time-based event logs" [19].

Summary of Phase 1 Deliverables

- **Persona & Scenario Definition:** We will document our target rider profile (e.g. Zomato delivery partner in Metro City) and illustrate typical work-weeks, earnings, and vulnerabilities (citing the stats above). This grounds our solution in reality.

- **Weekly Pricing Model:** We'll detail the weekly premium tiers and corresponding coverage limits, justified by the earnings data. For example, "Basic (₹30/week) covers ₹500/week loss; Premium (₹100/week) covers up to ₹2,000/week," aligning with average gig incomes (~₹800–1,500/day [3] [4]).

- **Parametric Triggers:** We'll list the chosen triggers (rainfall, temp, AQI, outages, curfews) and specify threshold values for each, with justification. E.g. "Rain $\geq$ 40 mm in 24h (heavy rain alert)" or "AQI $\geq$ 300 (severe pollution)". We'll cite industry practice like the WEF note that parametric payouts use such measurable indices [14].

- **AI/ML Integration Plan:** We'll outline where AI fits: premium pricing, risk scoring per region, fraud/anomaly detection (using models discussed above), and even predictive analytics (forecasting disruption probability). We'll name tools (e.g. Python, scikit-learn/TensorFlow for models).

- **Tech Stack:** We'll choose a stack such as a mobile/web front-end (React Native or Flutter for riders), a Node.js/Python backend, and a database (Firebase/MongoDB). APIs: OpenWeatherMap (weather/AQI), MapMyIndia/HERE (traffic), simulated platform API, and Razorpay sandbox for payments.

- **Repository & Presentation:** We will create a GitHub repo containing this README strategy document. The README.md will clearly cover the above points. We'll also prepare a 2-minute pitch video illustrating the idea flow (problem → solution → demo plan) with slides or sketches of the app UI and data flow.

In sum, **Phase 1** will produce a well-researched "Idea Document" detailing *who* we help (delivery riders), *what* risks we cover (income loss triggers), *how* the weekly insurance model works, and *what* technology we will use, all backed by data and citations. This sets a solid foundation before any coding begins.

Sources: We draw on recent industry reports and studies on gig-worker earnings [1] [2] [3], research on parametric insurance [13] [14], and technical docs for APIs and AI methods [16] [17] [19]. Each element of our plan is grounded in these findings.

---

[1] [5] [6] How much do delivery partners actually earn? A look inside the pay model for gig workers on Zomato, Swiggy
https://www.moneycontrol.com/news/business/startup/how-much-do-delivery-partners-actually-earn-a-look-inside-the-pay-model-for-gig-workers-on-zomato-swiggy-13756512.html

[2] [4] [7] [8] How much does a gig worker really earn? It's not quite what Deepinder Goyal would have us believe | Mint
https://www.livemint.com/economy/india-gig-workers-zomato-swiggy-earnings-real-pay-social-security-11767694201605.html

[3] [10] 10-Minute Delivery in Relentless Rains: What it Costs Riders and What They Earn?
https://www.thequint.com/jobs/heavy-rainfall-10-minute-delivery-blinkit-swiggy-instamart-zepto-delivery-persons-minimum-wages-insurance-latest-news

[9] [12] How air pollution is holding back India’s economy - LSE Business Review
https://blogs.lse.ac.uk/businessreview/2025/12/12/how-air-pollution-is-holding-back-indias-economy/

[11] [14] How parametric insurance is building climate resilience | World Economic Forum
https://www.weforum.org/stories/2025/01/what-is-parametric-insurance-and-how-is-it-building-climate-resilience/

[13] [15] Parametric protection for gig workers – The Parametric Post Issue 42 | InsTech
https://www.instech.co/knowledge-centre/parametric-protection-for-gig-workers-the-parametric-post-issue-42/

[16] Weather API
https://openweathermap.org/api

[17] MapmyIndia- India’s leader in premium quality digital map data, APIs, Smart City Solutions, GPS navigation, telematics, location-based SaaS and GIS Solutions
https://www.mapmyindia.com/traffic-analytics/

[18] API Sandbox Setup | Razorpay Docs
https://razorpay.com/docs/api/sandbox-setup/?preferred-country=US

[19] Geofencing for fraud prevention and mobile banking
https://radar.com/industry/insurance

[20] [21] Insurance claims estimation and fraud detection with optimized deep learning techniques - PMC
https://pmc.ncbi.nlm.nih.gov/articles/PMC12297633/